Parser with Flex and Bison

Gandem Nitin 2025MCS2152
Chirag Suthar 2025MCS2105

1 Introduction

This technical report describes the design and implementation of a mini programming language interpreter developed using **Flex** and **Bison**. The project demonstrates the complete front-end pipeline of a compiler-like system, including lexical analysis, syntax analysis, abstract syntax tree (AST) construction, semantic analysis, scoped symbol table management, and partial execution through constant evaluation.

The language is intentionally minimal yet expressive enough to illustrate key compiler construction concepts such as operator precedence, block scoping, semantic validation, and optimization via constant folding.

2 Description of the Designed Language

The designed language is a small, imperative, block-structured programming language inspired by C-like syntax. It supports integer-based computation and structured control flow constructs.

2.1 Supported Features

The language provides the following constructs:

- Variable declarations with optional initialization
- Assignment statements
- Arithmetic expressions with unary and binary operators
- Relational and equality expressions
- Conditional statements (`if--else`)
- Iterative statements (`while`)
- Nested block scoping using braces

2.2 Example Program

```
var x = 1 + 2 * 3;
var y;
y = x + 4;

if (y > 5) {
    y = y - 1;
}
```

2.3 Execution Model

The language follows a static execution model where:

- Variables are scoped lexically
- All values are integers

- Only constant expressions are evaluated at compile time
- Non-constant control flow is skipped safely

The system performs semantic analysis rather than full runtime interpretation.

3 Lexer Rules and Tokenization Strategy

Lexical analysis is implemented using **Flex**. The lexer converts raw source code into a stream of tokens consumed by the parser.

3.1 Token Categories

3.1.1 Keywords

- `var`
- `if`
- `else`
- `while`

Each keyword is mapped to a dedicated token.

3.1.2 Identifiers

Identifiers follow the rule:

$$[a\text{-}zA\text{-}Z\_][a\text{-}zA\text{-}Z0\text{-}9\_]*$$

Identifier names are dynamically allocated and passed to the parser using `yylval`.

3.1.3 Integer Literals

Integer literals consist of one or more digits:

$$[0\text{-}9]+$$

They are converted using `atoi` and stored as integer semantic values.

3.1.4 Operators and Delimiters

The lexer recognizes:

- Multi-character operators: `==`, `!=`, `<=`, `>=`
- Single-character operators and delimiters returned as literal tokens:

$$+ \ - \ * \ / \ < \ > \ = \ (\ ) \ \{ \ \} \ ;$$

This approach simplifies grammar design.

3.2 Comment Handling

Both single-line and multi-line comments are supported. A separate lexer state is used to handle block comments and detect unterminated comment errors.

3.3 Lexer Error Handling

Any unrecognized character results in an immediate lexer error with line number reporting, preventing undefined behavior.

4 Grammar Design and Operator Precedence

The grammar is implemented using **Bison** and is designed to be unambiguous and precedence-aware.

4.1 Operator Precedence

Operator precedence is enforced using a layered grammar structure:

Grammar Level	Operators
Primary	Literals, identifiers, parentheses
Unary	`+`, `-`
Multiplicative	`*`, `/`
Additive	`+`, `-`
Comparison	`<`, `>`, `<=`, `>=`
Equality	`==`, `!=`

This ensures correct parsing of expressions such as:

$$1 + 2 * 3 = 1 + (2 * 3)$$

4.2 Dangling Else Resolution

The dangling-else ambiguity is resolved using precedence declarations, ensuring that each `else` binds to the nearest unmatched `if`.

4.3 AST-Oriented Grammar

Each grammar rule constructs exactly one AST node, ensuring a clear separation between syntax and semantics.

5 AST Structure and Construction

The Abstract Syntax Tree (AST) is the central internal representation of the program.

5.1 AST Node Types

Each AST node contains:

- A node type identifier
- Source line number
- A union containing node-specific data

Supported node types include:

- Program and block nodes
- Variable declarations and assignments
- Conditional and loop statements
- Binary and unary expressions
- Integer literals and identifiers

5.2 AST Construction

AST nodes are created using dedicated constructor functions such as:

```
ast_make_binop()
ast_make_assign()
ast_make_if()
```

This centralizes memory allocation and simplifies debugging.

5.3 Constant Folding

An optimization pass traverses the AST to fold constant expressions at compile time. For example:

$$1 + 2 * 3 \rightarrow 7$$

Only safe expressions are folded, with division-by-zero checks enforced.

6 Error Handling Strategy

Error handling is implemented at multiple stages.

6.1 Lexical Errors

- Invalid characters
- Unterminated block comments

6.2 Syntax Errors

- Invalid grammar constructs
- Incorrect assignment targets

Parser errors include precise line numbers and descriptive messages.

6.3 Semantic Errors

Semantic analysis detects:

- Use of undeclared variables
- Variable redeclaration
- Assignment to undeclared variables
- Division by zero

Multiple semantic errors can be reported in a single execution.

7 Limitations and Possible Extensions

7.1 Current Limitations

- Only integer data type is supported
- No full runtime execution of non-constant expressions
- No functions or procedures
- No input/output facilities

7.2 Possible Extensions

- Full interpreter with runtime execution
- Additional data types such as floats and booleans
- Function definitions and call stack management
- Static type checking
- Code generation to assembly or bytecode

8 Conclusion

This project successfully demonstrates a complete compiler-style front end using Flex and Bison. It incorporates robust lexical and syntactic analysis, a well-designed AST, semantic validation through scoped symbol tables, and compile-time optimization via constant folding.

The system provides a strong foundation for extending the language into a full interpreter or compiler.